

后台管理模块 (Admin) 详细报告

一、需求与设计

1.1 功能需求

需求编号	需求描述	优先级
ADMIN-01	仪表盘统计 (用户数/消息数/群数/动态数/日活/周新/日消息/禁用数)	P1
ADMIN-02	用户管理 (搜索/角色变更/禁用启用)	P1
ADMIN-03	群组管理 (查看/解散)	P1
ADMIN-04	动态管理 (查看/删除)	P2
ADMIN-05	审计日志 (所有管理员操作自动记录)	P1
ADMIN-06	分页浏览 (所有列表支持分页)	P1
ADMIN-07	管理员权限验证	P0

1.2 设计决策

- 1. **自防护** — 管理员不能禁用自己
- 2. **全审计** — 每个写操作 (角色变更/禁用/解散群/删动态) 自动记录审计日志
- 3. **级联删除** — 解散群组时级联删除 reactions → visibility → messages → invitations → members → group → conversation (完整依赖顺序)
- 4. **软删除动态** — 动态删除设置 `deleted=1`，不物理删除
- 5. **分页封装** — 所有列表返回 `AdminPage<T> {rows, page, size, total}`

二、后端设计

2.1 核心文件清单

文件	行数	位置
AdminResource.java	106	src/main/java/com/example/chat/admin/AdminResource.java
AdminService.java	97	src/main/java/com/example/chat/admin/AdminService.java
AdminDao.java	189	src/main/java/com/example/chat/admin/AdminDao.java
DashboardStats.java	14	src/main/java/com/example/chat/admin/DashboardStats.java
AdminPage.java	12	src/main/java/com/example/chat/admin/AdminPage.java
AdminUserRow.java	14	src/main/java/com/example/chat/admin/AdminUserRow.java
AdminGroupRow.java	14	src/main/java/com/example/chat/admin/AdminGroupRow.java

文件	行数	位置
AdminMomentRow.java	17	src/main/java/com/example/chat/admin/AdminMomentRow.java
AdminAuditLogRow.java	16	src/main/java/com/example/chat/admin/AdminAuditLogRow.java
AdminDisableRequest.java	5	src/main/java/com/example/chat/admin/AdminDisableRequest.java
AdminRoleRequest.java	5	src/main/java/com/example/chat/admin/AdminRoleRequest.java

2.2 REST API 端点

权限验证： 所有端点调用 `SessionSupport.requireAdmin(request)` 验证管理员身份 代码位置：
`src/main/java/com/example/chat/common/SessionSupport.java` — `requireAdmin()` 方法

HTTP	路径	功能
GET	<code>/api/admin/dashboard</code>	仪表盘统计
GET	<code>/api/admin/users?q=&page=1&size=12</code>	用户列表 (支持搜索)
PUT	<code>/api/admin/users/{id}/role</code>	修改用户角色
PUT	<code>/api/admin/users/{id}/disable</code>	禁用/启用用户
GET	<code>/api/admin/groups?page=1&size=20</code>	群组列表
DELETE	<code>/api/admin/groups/{conversationId}</code>	解散群组
GET	<code>/api/admin/moments?page=1&size=20</code>	动态列表
DELETE	<code>/api/admin/moments/{id}</code>	删除动态
GET	<code>/api/admin/audit-logs?page=1&size=30</code>	审计日志

代码位置：`src/main/java/com/example/chat/admin/AdminResource.java`

2.3 DashboardStats — 仪表盘数据

```
public record DashboardStats(  
    long totalUsers,           // 总用户数  
    long totalMessages,       // 总消息数  
    long totalGroups,         // 总群组数  
    long totalMoments,        // 总动态数  
    long activeUsersToday,    // 今日活跃用户  
    long newUsersThisWeek,    // 本周新用户  
    long messagesToday,       // 今日消息数  
    long disabledUsers        // 禁用用户数  
)
```

所有8项统计在一个带子查询的SQL中完成，无需Java端聚合。

代码位置：

- Record: `src/main/java/com/example/chat/admin/DashboardStats.java`
- SQL: `src/main/java/com/example/chat/admin/AdminDao.java — dashboard()`

2.4 AdminService — 核心业务逻辑

方法	功能	审计操作
<code>dashboard()</code>	获取仪表盘统计	—
<code>users(q, page, size)</code>	用户列表 (支持搜索)	—
<code>setUserRole(adminId, userId, role)</code>	修改角色 (ADMIN/USER)	SET_USER_ROLE
<code>setUserDisabled(adminId, userId, disabled)</code>	禁用/启用 (防止自禁)	DISABLE_USER / ENABLE_USER
<code>groups(page, size)</code>	群组列表	—
<code>disbandGroup(adminId, conversationId)</code>	解散群 (级联删除)	DISBAND_GROUP
<code>moments(page, size)</code>	动态列表	—
<code>deleteMoment(adminId, momentId)</code>	删除动态 (软删除)	DELETE_MOMENT
<code>auditLogs(page, size)</code>	审计日志	—

代码位置: `src/main/java/com/example/chat/admin/AdminService.java`

2.5 AdminDao — 级联解散群

解散群的SQL执行顺序 (严格按照依赖关系):

```
1. DELETE message_reactions WHERE message_id IN (SELECT id FROM messages WHERE conversation_id = ?)
2. DELETE message_visibility WHERE message_id IN (SELECT id FROM messages WHERE conversation_id = ?)
3. UPDATE messages SET reply_to_message_id = NULL WHERE conversation_id = ?
4. DELETE messages WHERE conversation_id = ?
5. DELETE chat_group_invitations WHERE group_id = ?
6. DELETE chat_group_members WHERE group_id = ?
7. DELETE conversation_members WHERE conversation_id = ?
8. DELETE chat_groups WHERE conversation_id = ?
9. DELETE conversations WHERE id = ?
```

代码位置: `src/main/java/com/example/chat/admin/AdminDao.java — disbandGroup()` 方法

2.6 审计日志

```
void logAction(Connection, long adminId, String action, String targetType, Long
targetId, String detail)
// INSERT INTO admin_audit_logs (admin_id, action, target_type, target_id, detail)
```

每次管理员写操作自动调用，记录：

- `admin_id` — 操作者
- `action` — 操作类型 (SET_USER_ROLE, DISABLE_USER, DISBAND_GROUP, DELETE_MOMENT 等)
- `target_type` — 操作目标类型 (USER, GROUP, MOMENT)
- `target_id` — 操作目标ID (可为NULL)
- `detail` — 操作详情 (如 "role:ADMIN", "disabled:true")

代码位置：`src/main/java/com/example/chat/admin/AdminDao.java` — `logAction()` 方法

三、数据库设计

3.1 admin_audit_logs 表

列名	类型	约束	说明
id	BIGINT	PK, AUTO_INCREMENT	日志ID
admin_id	BIGINT	FK → users, NOT NULL	管理员ID
action	VARCHAR(40)	NOT NULL	操作类型
target_type	VARCHAR(40)	—	目标类型
target_id	BIGINT	—	目标ID
detail	VARCHAR(500)	—	详情
created_at	TIMESTAMP	NOT NULL	操作时间

建表SQL：`src/main/resources/schema.sql` 第241-250行 SchemaMigrator：
`src/main/java/com/example/chat/config/SchemaMigrator.java` — `ensureAdminAuditLogsTable()`

3.2 涉及的其他表

admin 模块查询/修改以下表 (非 DDL，读写操作)：

- `users` — 查询、更新(role/disabled)
- `messages` — 查询、删除
- `chat_groups` — 查询、删除
- `chat_group_members` — 删除
- `chat_group_invitations` — 删除
- `conversation_members` — 删除
- `conversations` — 删除
- `message_reactions` — 删除
- `message_visibility` — 删除
- `moments` — 查询、更新(deleted)

- `moment_media` — 子查询
- `moment_likes` — 子查询
- `moment_comments` — 子查询

四、前端设计

4.1 页面布局

```
<div id="adminView">
  <nav> <!-- Tab导航：总览 / 用户管理 / 群组管理 / 动态管理 / 审计日志 --> </nav>
  <section id="adminOverview"> <!-- 仪表盘 8张统计卡片 --> </section>
  <section id="adminUsers"> <!-- 用户表格（搜索+分页） --> </section>
  <section id="adminGroups"> <!-- 群组表格 --> </section>
  <section id="adminMoments"> <!-- 动态表格 --> </section>
  <section id="adminLogs"> <!-- 审计日志表格 --> </section>
</div>
```

代码位置：`src/main/webapp/app.html` — `#adminView` 区域

4.2 核心JS函数

函数	功能	位置 (admin.js行号)
<code>loadAdminDashboard()</code>	并行加载5个API (总览+用户+群+动态+日志)	~136
<code>renderAdminStats(stats)</code>	渲染8张统计卡片	~14
<code>renderAdminUsers(page)</code>	渲染用户表格+分页+角色/禁用按钮	~35
<code>renderAdminGroups(page)</code>	渲染群组表格+解散按钮	~66
<code>renderAdminMoments(page)</code>	渲染动态表格+删除按钮	~90
<code>renderAdminLogs(page)</code>	渲染审计日志表格	~114
<code>selectAdminTab(tab)</code>	Tab切换	~155

代码位置：`src/main/webapp/assets/js/admin.js` (214行)

4.3 事件委托

所有操作通过 `document click` 委托 + `data-*` 属性分发：

- `data-admin-tab` — 切换Tab
- `data-admin-refresh` — 刷新Tab
- `data-admin-users-page` — 用户分页
- `data-set-role` + `data-user-id` + `data-role` — 角色切换
- `data-toggle-disable` + `data-user-id` + `data-disabled` — 禁用/启用切换
- `data-disband-group` + `data-conversation-id` — 解散群组
- `data-delete-moment` + `data-moment-id` — 删除动态

代码位置：`src/main/webapp/assets/js/admin.js` — 事件监听器区域

4.4 权限控制

- 非 ADMIN 用户：导航栏中 "后台管理" 按钮隐藏
- 管理员：显示后台管理入口，前端 `loadAdminDashboard()` 仅在 `AppState.me.role === "ADMIN"` 时调用
- 后端双重保障：所有 `/api/admin/*` 端点通过 `SessionSupport.requireAdmin()` 验证

代码位置：

- 前端入口：`src/main/webapp/assets/js/app.js` — `boot()` 中的角色检查
- 后端守卫：`src/main/java/com/example/chat/common/SessionSupport.java` — `requireAdmin()`
- 后端资源：`src/main/java/com/example/chat/admin/AdminResource.java` — 每个方法首行调用 `requireAdmin()`